实验一　Java 基础语法实验报告

班级：__计算机 25-1__　学号：_25034480138_　姓名：___吴佳鑫____　成绩：_____

一、实验目的

熟练掌握 Java 语言的基本语法，包括注释、标识符、数据类型、常量、变量、运算符和程序控制结构。

二、实验内容与程序实现

本次实验共 6 道题，均采用 static 方法实现，在 main 方法中依次调用。以下逐题给出解题思路、主要程序代码和运行结果截图。

1. 打印 100~200 之间的素数

题目：编写一个程序，打印 100~200 之间的素数，要求每行按 10 个数（数与数之间有一个空格隔离）的形式对其输出。

（1）解题思路

判定素数：对整数 n，用变量 i 从 n−1 递减试探到 2，只要存在 $n \% i == 0$，说明 n 有除 1 和自身以外的因子，返回 false；循环结束仍没有整除因子则返回 true。判定逻辑单独封装为 IsPrime(int n)。

输出格式：PrintPrime() 中用 for 循环遍历 100~200（$i < 201$），对每个素数调用 System.out.print(i) 输出，并用计数器 count 记录本行已输出的个数；当 count 达到 10 时清零并 println() 换行，否则输出一个空格作为数与数之间的分隔。

结果核对：100~200 之间共 21 个素数，因此输出为 3 行，前两行各 10 个、最后一行 1 个（199），与图 1 一致。

（2）主要程序代码

```
public static boolean IsPrime(int n)
{
    int i = n - 1;
    while(i > 1)
    {
        if(n % i == 0)
            return false;
        i -= 1;
    }
    return true;
}

public static void PrintPrime()
{
    int count = 0;
    for(int i = 100;i < 201;i++)
    {
        if(IsPrime(i))
        {
            System.out.print(i);
```

```
            count++;
            if(count == 10)
            {
                count = 0;
                System.out.println();
            }
            else
            {
                System.out.print(" ");
            }
        }
    }
    System.out.println();
}
```

（3）测试截图

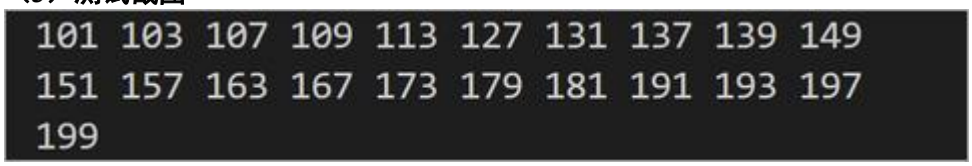


图 1　程序输出 100~200 之间的全部素数，每行 10 个、数间以空格分隔，共 21 个素数。

2. 用 if 与 switch 两种方式计算分段函数 y

题目：编写一个程序，给定一个 t 的值（可初始化定义），按下式计算 y 值并输出，要求分别写出 if 和 switch 语句。

（1）解题思路

分段函数为：0 ≤ t < 1 时 $y = t^2 - 1$；1 ≤ t < 3 时 $y = t^3 - 2t - 2$；3 ≤ t < 5 时 $y = t^2 - t \cdot \sin(t)$；5 ≤ t < 7 时 y = t + 1；其余情况 y = t − 1。

if 实现（CalculateByIf）：按区间从上到下用 if / else if / else 逐级判断，命中哪个区间就用对应表达式给 result 赋值，最后返回 result；else 分支负责 t < 0 或 t ≥ 7 的情况。

switch 实现（CalculateBySwitch）：switch 不能直接对 double 分支，先用 int key = (int)Math.floor(t) 把 t 映射为所在区间的整数键，再用 Java 14 之后的箭头标签写法，以 case 0、case 1,2、case 3,4、case 5,6 合并同一区间的多个键值，default 处理其余区间。

结果核对：两种写法对同一 t 应得到相同 y。例如测试点 t = 3.3 落在 3 ≤ t < 5，$y = 3.3^2 - 3.3 \cdot \sin(3.3) \approx 11.410560790672719$；另一个测试点输出 7.5，与图 2 一致。

（2）主要程序代码

```
public static double CalculateByIf(double t)
{
    double result = 0.0;
    if(0 <= t && t < 1 )
    {
        result = t * t - 1;
    }
    else if(1 <= t && t < 3)
    {
        result = t * t * t  - 2 * t - 2;
    }
    else if(3 <= t && t < 5)
    {
        result = t * t - t * Math.sin(t);
    }
```

```
    else if(5 <= t && t < 7)
    {
        result = t + 1;
    }
    else
    {
        result = t - 1;
    }
    return result;
}

public static double CalculateBySwitch(double t)
{
    double result = 0.0;
    int key = (int)Math.floor(t);
    switch(key)
    {
        case 0 ->
            result = t * t - 1;
        case 1,2 ->
            result = t * t * t - 2 * t - 2;
        case 3,4 ->
            result = t * t - t * Math.sin(t);
        case 5,6 ->
            result = t + 1;
        default ->
            result = t - 1;
    }
    return result;
}
```

（3）测试截图



图 2　分别对两个测试点计算得到的 y 值：11.410560790672719 与 7.5，与分段函数手算结果一致。

3. 打印所有“水仙花数”

题目：打印出所有的“水仙花数”。“水仙花数”是指一个三位数，其各位数字的立方和等于该数本身。

（1）解题思路

用 for 循环枚举全部三位数 i = 100~999（i < 1000），保证不重不漏。

利用整数除法与取模拆分各位：百位 number1 = i / 100，个位 number3 = i % 10，十位 number2 = (i / 10) % 10。

判断 number1³ + number2³ + number3³ 是否等于 i，相等即为水仙花数并打印。最终得到 153、370、371、407 四个数，与图 3 一致。

（2）主要程序代码

```
// 3.打印出所有的“水仙花数”。
// “水仙花数”是指一个三位数，其各位数字的立方和等于该数本身
public static void PrintNarcissisticNumber()
{
    int number1 = 0;
    int number2 = 0;
```

```
    int number3 = 0;
    for(int i = 100;i < 1000;i++)
    {
        number1 = i / 100;
        number3 = i % 10;
        number2 = (i / 10) % 10;
        if(number1 * number1 * number1 + number2 * number2 * number2 + number3 *
number3 * number3 == i)
            System.out.println(i);
    }
}
```

（3）测试截图



图 3　程序打印出的全部水仙花数：153、370、371、407。

4. 随机生成 n 个 0~100 的数并求和

题目：编写一个 Java 应用程序，先产生 1 个随机数 n（0 < n < 10），然后随机生成 n 个 0~100 的随机数，输出这 n 个随机数的和。

（1）解题思路

用 Random r = new Random(67) 创建随机数生成器并固定种子，使每次运行的结果可复现，便于测试核对。

n = r.nextInt(9) + 1 产生 1~9 的整数，满足 0 < n < 10；再用 for 循环执行 n 次，每次 r.nextInt(101) 产生 0~100 的随机整数并累加到 result。

方法返回累加和，本次运行输出 165（见图 4）。若去掉固定种子，每次运行会得到不同的和。

（2）主要程序代码

```
// 4.编写一个 Java 应用程序，先产生 1 个随机数 n（0<n<10）
// 然后随机生成 n 个 0~100 的随机数，输出这 n 个随机数的和
public static int RandomNumberTotal()
{
    int result = 0;
    Random r = new Random(67);
    int n = r.nextInt(9) + 1;
    for(int i = 0;i < n;i++)
    {
        result += r.nextInt(101);
    }
    return result;
}
```

（3）测试截图



图 4　本次运行得到的 n 个 0~100 随机数之和：165（Random(67) 固定种子，结果可复现）。

5. 求 1000 之内的所有完全数

题目：求 1000 之内的所有完全数。完全数就是一个数恰好等于它的因子之和。例如 6 的因子为 1, 2, 3，而 6 = 1 + 2 + 3，因此 6 是一个完全数。

（1）解题思路

外层 for 循环遍历 i = 1~999，对每个 i 用 total 记录其真因子（不含自身）之和。

内层用 j 从 i − 1 递减到 1（while(j > 0)），若 i % j == 0 则把 j 累加进 total，循环结束即得到全部真因子之和。

若 i == total 则 i 为完全数并打印。1000 之内共有 6、28、496 三个完全数，与图 5 一致。

（2）主要程序代码

```
// 5.求 1000 之内的所有完全数。完全数就是一个数恰好等于它的因子之和
// 例如 6 的因子为 1,2,3，而 6=1+2+3，因此 6 是一个完全数
public static void PerfectNumber()
{
    for(int i = 1;i < 1000;i++)
    {
        int total = 0;
        int j = i - 1;
        while(j > 0)
        {
            if(i % j == 0)
                total += j;
            j -= 1;
        }
        if(i == total)
            System.out.println(i);
    }
}
```

（3）测试截图



图 5　1000 之内的全部完全数：6、28、496。

6. 按精度要求求 e 的近似值

题目：编程求出 e = 1 + 1/1! + 1/2! + … + 1/n! + … 的近似值，当 1/n! 小于 0.0001 时停止计算。

（1）解题思路

不直接计算阶乘（容易溢出），而用递推生成每一项：设当前项 number = 1/0! = 1.0，count = 1；每轮先把 number 加到 e 中，再用 number /= count++ 得到下一项 1/1!、1/2!、1/3! …，从而用一次除法完成阶乘递推。

循环条件 while(number > 0.0001) 在累加之前判断，因此当某项已小于 0.0001 时不再加入，恰好满足“当 1/n! 小于 0.0001 时停止计算”。最后累加的是 1/7!，即 e ≈ 1 + 1 + 1/2! + … + 1/7!。

输出 2.7182539682539684（图 6），与数学常数 e ≈ 2.7182818 相比误差约 2.8×10^{-5}，符合题目给定的精度要求。

（2）主要程序代码

```
// 6.编程求出 e=1+1/1!+1/2!+...+1/n!+...的近似值，当 1/n!小于 0.0001 时停止计算。
public static double CalculateE()
{
    double number = 1.0;
    double e = 0.0;
    int count = 1;
    while(number > 0.0001)
    {
        e += number;
        number /= count++;
    }
    return e;
}
```

（3）测试截图



图 6　按 1/n! < 0.0001 停止条件算出的 e 近似值：2.7182539682539684。

三、小结

本次实验围绕 Java 基础语法展开，6 道题分别练习了方法定义与返回值、变量与基本数据类型、算术与逻辑运算符、while / for 循环、if-else 与 switch 多分支、Math 与 Random 常用类，以及 System.out 标准输出。所有题目均以 static 方法封装，在 main 中调用，结构清晰、便于逐个测试。

主要收获：
① 用递推式 number /= count++ 生成级数各项，比反复计算阶乘更高效，也避免了 int 阶乘溢出；
② switch 通过 Math.floor(t) 把连续区间映射成离散整数键，配合箭头标签与多值 case，可以把冗长的 if-else 改写成更紧凑的分支结构；
③ 用计数变量 count 控制每行输出个数，是格式化控制台输出的常用手法；
④ 给 Random 指定固定种子，能让随机数相关的测试结果可复现；
⑤ 素数、水仙花数、完全数三题本质上都是“枚举 + 条件判定”，拆位用整除与取模，判因子用取模试除，思路可以互相迁移。

测试结果：6 道题的运行结果分别为 ：
100~200 的 21 个素数、两个测试点的 y 值 11.410560790672719 与 7.5、四个水仙花数 153/370/371/407、随机和 165、三个完全数 6/28/496、e ≈ 2.7182539682539684，均与理论预期一致，说明程序逻辑正确。

不足与改进：IsPrime 从 n−1 开始递减试除、PerfectNumber 从 i−1 开始枚举因子，时间复杂度偏高，可把试除范围缩到 √n（如 experiment1.java 中的写法）以提升效率；switch 版本依赖区间端点为整数，若分段边界改为小数则需换回 if 判断；随机数题目前只输出总和，可以补充打印每一个随机数及 n 的值，让测试过程更直观。

源代码 github 仓库：https://github.com/wjx-nahida/java-course-experiment.git